Part 1: Partial Derivatives (7 points)

For the first three problems, define

$$L = \sum_{i=1}^{N} |\mu - x(i)| + \lambda\mu^2, \quad \mu = \theta^3,$$

where all variables involved are real-valued.

For the last problem, we consider the function softmax which maps a vector $z \in \mathbb{R}^k$ to the vector $\text{softmax}(z) \in \mathbb{R}^k$ defined by

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}}.$$

The entries of this vector are positive and sum to 1, so we can view $\text{softmax}(z)$ as a probability distribution over the set $\{1, \ldots, k\}$.

(a) Compute the derivative $\partial L/\partial x(i)$. Leaving all other variables fixed, what value of $x(i)$ minimizes L? (1.5 points)

(b) Compute the derivative $\partial L/\partial \mu$. If $\lambda = 0$, what values of μ minimize L? (2 points)

(c) Compute the derivative $\partial L/\partial \theta$. (1.5 points)

(d) Compute

$$\frac{\partial}{\partial z_i} \ln \text{softmax}(z)_j.$$

(Hint: first define $Z = \ln \sum_{j=1}^{k} e^{z_j}$ and compute $\partial Z/\partial z_j$.) (2 points)

Part 2: Linear Regression (7 points)

For this part you'll use `data/powerplant.csv`, a dataset from a combined cycle power plant. We consider the problem of predicting y (output power in megawatts) as a function of x (ambient air temperature in centigrade). You can load empirical values of these variables, represented as numpy arrays, with `x, y = iml.load_powerplant()`.

Write $(x(i), y(i))$ for our datapoints. Given predictions $\hat{y}(i)$ meant to approximate $y(i)$, the associated **mean squared error** is

$$L = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}(i) - y(i))^2.$$

When the predictions $\hat{y}(i)$ are linear functions of the model parameters, minimizing the mean squared error is called a **linear least squares problem**.

We will write $\theta = (\theta_0, \ldots, \theta_{P-1})$ for the vector of parameters. Remember that the *gradient* ∇L refers to the vector of partial derivatives with respect to the parameters, namely

$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial \theta_0} & \cdots & \frac{\partial L}{\partial \theta_{P-1}} \end{bmatrix}.$$

(a) Consider the linear model, defined by parameters (θ_0, θ_1), which predicts

$$\hat{y}(i) = \theta_0 + \theta_1 x(i)$$

for each index i. Express the gradient ∇L of mean squared error as a function of the parameter vector θ and the datapoints $(x(i), y(i))$. (3 points)

(b) Extend the `SimpleRegressor` class from `iml/simple_regressor.py`, implementing the method `fit(self, x, y, lr, steps)` to implement gradient descent on mean squared error. Run `fit()` on the *unnormalized* data vectors `x` and `y`, and show the training process over 20 iterations with `SimpleRegressor.show_training()`. Find a learning rate that makes the loss decrease. (2 points)

(c) Now, normalize your data in some way and run `fit()` and `show_training()` again, ensuring that you converge to an acceptable model in 20 iterations. (2 points)

Part 3: k-Means and PCA (6 points)

For the this part you'll need the iris dataset and `data/ulu.csv`, which you can load with `iml.load_iris()` and `iml.load_ulu()` respectively.

(a) Suppose we didn't have access to the species attribute of the iris dataset. Implement the Lloyd-Forgy algorithm for k-means and use it to infer a partition of the iris dataset into three species. Compute the confusion matrix and the accuracy of the partition relative to the true species. (3 points)

(b) Investigate `data/ulu.csv` using PCA and explain how this dataset was produced. Using scikit-learn, run both k-means clustering and DBSCAN on this dataset. Which method produces more reasonable "clusters"? (3 points)